

Uzbekistan OS

Technical Design Document (TDD)

Version 1.0

1. Purpose

Defines the technical architecture for the Uzbekistan OS MVP, including frontend, backend, AI orchestration, RAG, infrastructure, APIs, database, deployment, and engineering standards.

2. High-Level Architecture

Components: Next.js frontend → API Gateway → FastAPI services → AI Orchestrator → RAG Engine → PostgreSQL + pgvector + Redis → Knowledge Base → Crawler Pipeline → Official Government Sources.

3. Technology Stack

Frontend: Next.js, React, TypeScript, Tailwind, shadcn/ui. Backend: FastAPI, Python, SQLAlchemy, Alembic. Database: PostgreSQL + pgvector. AI: OpenAI Responses API, embeddings, function calling. Infrastructure: Docker, Nginx, GitHub Actions, S3-compatible storage.

4. Repository Structure

apps (web, api), packages (ai, database, shared, crawler, rag), docs, data, scripts, docker.

5. Core Components

Web App, API Server, AI Orchestrator, RAG Engine, Crawler Pipeline, Background Workers.

6. AI Request Lifecycle

User → Chat API → AI Orchestrator → Intent Detection → Retrieval → Prompt Builder → LLM → Validation → Source Attribution → Response.

7. RAG Pipeline

Hybrid retrieval: query normalization, metadata filtering, vector search, keyword search, reranking, context compression, prompt assembly, citations.

8. Knowledge Architecture

Raw documents are parsed into structured JSON, validated, embedded, indexed, and stored before being used for retrieval.

9. Knowledge Domains

Immigration, Tourism, Business, Healthcare, Banking, Transportation, Housing, Education, Legal, Taxation.

10. Prompt Architecture

System prompt + user profile + conversation memory + retrieved knowledge + user query + output instructions.

11. API Design

Versioned REST endpoints: /chat, /search, /documents, /workflow, /sources, /auth, /profile, /feedback.

12. Authentication

Guest sessions, Google OAuth, email/password, JWT authentication.

13. Caching

Redis caches sessions, conversations, embeddings, and frequently requested responses.

14. Logging & Observability

Capture request IDs, user IDs, latency, retrieved documents, model usage, tokens, and feedback.

15. Security

HTTPS, rate limiting, validation, prompt-injection defenses, SQL injection protection, encrypted secrets, audit logging.

16. Deployment

Docker Compose stack including Nginx, Web, API, PostgreSQL, Redis, Object Storage, and Background Workers.

17. Scalability

Independent horizontal scaling of web, API, AI workers, crawler, embedding workers, PostgreSQL replicas, and Redis.

18. Future Services

OCR, voice interface, government integrations, autonomous agents, notifications, analytics, CMS, translation pipeline, knowledge approval workflow.

Recommended Next Documents

Database Design Specification, Knowledge Schema Specification, OpenAPI Specification, Prompt & AI Specification, Frontend UX Specification, and Architecture Decision Records (ADRs).